```go
package repositories

import (
"hospital-hr-system/internal/models"

"gorm.io/gorm"
)

// EmployeeRepositoryInterface defines employee repository methods
type EmployeeRepositoryInterface interface {
Create(employee *models.Employee) error
FindByID(id uint) (*models.Employee, error)
FindByEmployeeNumber(employeeNumber string) (*models.Employee, error)
FindByEmail(email string) (*models.Employee, error)
Update(employee *models.Employee) error
Delete(id uint) error
List(offset, limit int, filters EmployeeFilters) ([]*models.Employee, int64, error)
FindByDepartment(departmentID uint, offset, limit int) ([]*models.Employee, int64,
error)
FindByPosition(positionID uint, offset, limit int) ([]*models.Employee, int64, error)
FindByStatus(status models.EmploymentStatus, offset, limit int) ([]*models.Employee,
int64, error)
SearchByName(searchTerm string, offset, limit int) ([]*models.Employee, int64, error)
ExistsByEmployeeNumber(employeeNumber string) (bool, error)
ExistsByEmail(email string) (bool, error)
CountByDepartment(departmentID uint) (int64, error)
CountByStatus(status models.EmploymentStatus) (int64, error)
GetEmployeeWithDetails(id uint) (*models.Employee, error)
UpdateStatus(id uint, status models.EmploymentStatus) error
}

// EmployeeFilters represents filtering options for employee queries
type EmployeeFilters struct {
DepartmentID *uint `json:"department_id"`
PositionID *uint `json:"position_id"`
Status *models.EmploymentStatus `json:"status"`
Gender *models.Gender `json:"gender"`
SearchTerm string `json:"search_term"`
}

// EmployeeRepository implements EmployeeRepositoryInterface
type EmployeeRepository struct {
db *gorm.DB
}
```

```
// NewEmployeeRepository creates a new employee repository
func NewEmployeeRepository(db *gorm.DB) EmployeeRepositoryInterface {
return &EmployeeRepository{db: db}
}

// Create creates a new employee
func (r *EmployeeRepository) Create(employee *models.Employee) error {
return r.db.Create(employee).Error
}

// FindByID finds an employee by ID with basic relationships
func (r *EmployeeRepository) FindByID(id uint) (*models.Employee, error) {
var employee models.Employee
err :=
r.db.Preload("Department").Preload("Position").Preload("ReportsTo").First(&employee
, id).Error
if err != nil {
return nil, err
}
return &employee, nil
}

//... more functions for repository

============================== Service
==================================
package services

import (
"errors"
"fmt"
"time"

"hospital-hr-system/internal/models"
"hospital-hr-system/internal/repositories"

"gorm.io/gorm"
)

// EmployeeService handles employee business logic
type EmployeeService struct {
employeeRepo repositories.EmployeeRepositoryInterface
userRepo repositories.UserRepositoryInterface
```

```
db *gorm.DB
}

// NewEmployeeService creates a new employee service
func NewEmployeeService(employeeRepo repositories.EmployeeRepositoryInterface,
userRepo repositories.UserRepositoryInterface, db *gorm.DB) *EmployeeService {
return &EmployeeService{
employeeRepo: employeeRepo,
userRepo: userRepo,
db: db,
}
}

// CreateEmployeeRequest represents create employee request
type CreateEmployeeRequest struct {
EmployeeNumber string `json:"employee_number" validate:"required,max=20"`
FirstName string `json:"first_name" validate:"required,max=50"`
LastName string `json:"last_name" validate:"required,max=50"`
MiddleName string `json:"middle_name,omitempty" validate:"max=50"`
Email string `json:"email" validate:"required,email,max=100"`
Phone string `json:"phone,omitempty" validate:"max=20"`
DateOfBirth *time.Time `json:"date_of_birth,omitempty"`
Gender *models.Gender `json:"gender,omitempty"`
MaritalStatus *models.MaritalStatus `json:"marital_status,omitempty"`
HireDate time.Time `json:"hire_date" validate:"required"`
DepartmentID uint `json:"department_id" validate:"required"`
PositionID uint `json:"position_id" validate:"required"`
ReportsToID *uint `json:"reports_to_id,omitempty"`

// Personal Details (optional during creation)
PersonalDetails *CreatePersonalDetailsRequest `json:"personal_details,omitempty"`

// Professional Details (optional during creation)
ProfessionalDetails *CreateProfessionalDetailsRequest
`json:"professional_details,omitempty"`

// Emergency Contacts (optional during creation)
EmergencyContacts []CreateEmergencyContactRequest
`json:"emergency_contacts,omitempty"`

// User Account (optional - for creating login access)
CreateUserAccount bool `json:"create_user_account,omitempty"`
Username string `json:"username,omitempty"`
Password string `json:"password,omitempty"`
```

```
}

// UpdateEmployeeRequest represents update employee request
type UpdateEmployeeRequest struct {
FirstName *string `json:"first_name,omitempty" validate:"omitempty,max=50"`
LastName *string `json:"last_name,omitempty" validate:"omitempty,max=50"`
MiddleName *string `json:"middle_name,omitempty" validate:"omitempty,max=50"`
Email *string `json:"email,omitempty" validate:"omitempty,email,max=100"`
Phone *string `json:"phone,omitempty" validate:"omitempty,max=20"`
DateOfBirth *time.Time `json:"date_of_birth,omitempty"`
Gender *models.Gender `json:"gender,omitempty"`
MaritalStatus *models.MaritalStatus `json:"marital_status,omitempty"`
DepartmentID *uint `json:"department_id,omitempty"`
PositionID *uint `json:"position_id,omitempty"`
ReportsToID *uint `json:"reports_to_id,omitempty"`
Status *models.EmploymentStatus `json:"status,omitempty"`
}

// CreateProfessionalDetailsRequest represents professional details creation
type CreateProfessionalDetailsRequest struct {
PreviousExperience int `json:"previous_experience,omitempty"`
Specialization string `json:"specialization,omitempty" validate:"max=100"`
Skills string `json:"skills,omitempty"`
Education string `json:"education,omitempty"`
ProfessionalLicense string `json:"professional_license,omitempty"` // Medical/nursing
license numbers
References string `json:"references,omitempty"`
BackgroundCheck bool `json:"background_check,omitempty"`
}

// CreateEmergencyContactRequest represents emergency contact creation
type CreateEmergencyContactRequest struct {
Name string `json:"name" validate:"required,max=100"`
Relationship string `json:"relationship,omitempty" validate:"max=50"`
Phone string `json:"phone" validate:"required,max=20"`
Email string `json:"email,omitempty" validate:"omitempty,email,max=100"`
Address string `json:"address,omitempty" validate:"max=255"`
IsPrimary bool `json:"is_primary,omitempty"`
}

// EmployeeResponse represents employee response data
type EmployeeResponse struct {
ID uint `json:"id"`
EmployeeNumber string `json:"employee_number"`
```

```
FirstName string `json:"first_name"`
LastName string `json:"last_name"`
MiddleName string `json:"middle_name,omitempty"`
FullName string `json:"full_name"`
Email string `json:"email"`
Phone string `json:"phone,omitempty"`
DateOfBirth *time.Time `json:"date_of_birth,omitempty"`
Age *int `json:"age,omitempty"`
Gender *models.Gender `json:"gender,omitempty"`
MaritalStatus *models.MaritalStatus `json:"marital_status,omitempty"`
HireDate time.Time `json:"hire_date"`
TerminationDate *time.Time `json:"termination_date,omitempty"`
Status models.EmploymentStatus `json:"status"`
Department *DepartmentInfo `json:"department,omitempty"`
Position *PositionInfo `json:"position,omitempty"`
ReportsTo *EmployeeBasicInfo `json:"reports_to,omitempty"`
DirectReports []EmployeeBasicInfo `json:"direct_reports,omitempty"`
HasUserAccount bool `json:"has_user_account"`
CreatedAt time.Time `json:"created_at"`
UpdatedAt time.Time `json:"updated_at"`
}

// EmployeeDetailResponse represents detailed employee response
type EmployeeDetailResponse struct {
*EmployeeResponse
PersonalDetails *PersonalDetailsResponse `json:"personal_details,omitempty"`
ProfessionalDetails *ProfessionalDetailsResponse
`json:"professional_details,omitempty"`
EmergencyContacts []EmergencyContactResponse
`json:"emergency_contacts,omitempty"`
}

// EmployeeBasicInfo represents basic employee info for relationships
type EmployeeBasicInfo struct {
ID uint `json:"id"`
EmployeeNumber string `json:"employee_number"`
FirstName string `json:"first_name"`
LastName string `json:"last_name"`
FullName string `json:"full_name"`
Email string `json:"email"`
}

// DepartmentInfo represents department information
type DepartmentInfo struct {
```

```
ID uint `json:"id"`
Name string `json:"name"`
Code string `json:"code"`
Description string `json:"description,omitempty"`
}

// PositionInfo represents position information
type PositionInfo struct {
ID uint `json:"id"`
Title string `json:"title"`
Code string `json:"code"`
Description string `json:"description,omitempty"`
Level int `json:"level"`
}

// ProfessionalDetailsResponse represents professional details response
type ProfessionalDetailsResponse struct {
PreviousExperience int `json:"previous_experience"`
Specialization string `json:"specialization,omitempty"`
Skills string `json:"skills,omitempty"`
Education string `json:"education,omitempty"`
ProfessionalLicense string `json:"professional_license,omitempty"`
References string `json:"references,omitempty"`
BackgroundCheck bool `json:"background_check"`
BackgroundCheckDate *time.Time `json:"background_check_date,omitempty"`
}

// EmergencyContactResponse represents emergency contact response
type EmergencyContactResponse struct {
ID uint `json:"id"`
Name string `json:"name"`
Relationship string `json:"relationship,omitempty"`
Phone string `json:"phone"`
Email string `json:"email,omitempty"`
Address string `json:"address,omitempty"`
IsPrimary bool `json:"is_primary"`
}

// GetEmployee gets an employee by ID
func (s *EmployeeService) GetEmployee(id uint) (*EmployeeResponse, error) {
employee, err := s.employeeRepo.FindByID(id)
if err != nil {
return nil, err
}
```

```
return s.mapToEmployeeResponse(employee), nil
}

//--- More functions for service

============================== Handler
=========================================
package handlers

import (
"encoding/json"
"net/http"
"strconv"

"hospital-hr-system/internal/middleware"
"hospital-hr-system/internal/models"
"hospital-hr-system/internal/repositories"
"hospital-hr-system/internal/services"
"hospital-hr-system/pkg/response"

"github.com/go-playground/validator/v10"
"github.com/gorilla/mux"
)

// EmployeeHandler handles employee-related HTTP requests
type EmployeeHandler struct {
employeeService *services.EmployeeService
validator *validator.Validate
}

// NewEmployeeHandler creates a new employee handler
func NewEmployeeHandler(employeeService *services.EmployeeService)
*EmployeeHandler {
return &EmployeeHandler{
employeeService: employeeService,
validator: validator.New(),
}
}

// CreateEmployee handles employee creation
func (h *EmployeeHandler) CreateEmployee(w http.ResponseWriter, r *http.Request)
{
var req services.CreateEmployeeRequest
if err := json.NewDecoder(r.Body).Decode(&req); err != nil {
```

```
response.BadRequest(w, "Invalid JSON format", err.Error())
return
}

// Validate request
if err := h.validator.Struct(&req); err != nil {
response.UnprocessableEntity(w, "Validation failed", err.Error())
return
}

// Create employee
employee, err := h.employeeService.CreateEmployee(&req)
if err != nil {
response.BadRequest(w, "Failed to create employee", err.Error())
return
}

response.Created(w, "Employee created successfully", employee)
}

//----More functions for handler

================================== Routes
==================================
package routes

import (
"net/http"

"hospital-hr-system/internal/handlers"
"hospital-hr-system/internal/middleware"

"github.com/gorilla/mux"
)

// SetupEmployeeRoutes sets up employee routes
func SetupEmployeeRoutes(router *mux.Router, employeeHandler
*handlers.EmployeeHandler, authMiddleware *middleware.AuthMiddleware) {
// All employee routes require authentication
employeeRouter := router.PathPrefix("/api/employees").Subrouter()
employeeRouter.Use(authMiddleware.RequireAuth)

// GET /api/employees - List employees (all authenticated users)
```

```
employeeRouter.HandleFunc("",
employeeHandler.ListEmployees).Methods(http.MethodGet)

// GET /api/employees/search - Search employees (all authenticated users)
employeeRouter.HandleFunc("/search",
employeeHandler.SearchEmployees).Methods(http.MethodGet)

// GET /api/employees/stats - Get employee statistics (admin/manager only)
employeeRouter.HandleFunc("/stats",
employeeHandler.GetEmployeeStats).Methods(http.MethodGet)

// GET /api/employees/generate-number - Generate employee number
(admin/manager only)
employeeRouter.HandleFunc("/generate-number",
employeeHandler.GenerateEmployeeNumber).Methods(http.MethodGet)

// POST /api/employees - Create employee (admin/manager only)
employeeRouter.Handle("",
authMiddleware.RequireRole("admin", "manager")
(http.HandlerFunc(employeeHandler.CreateEmployee)),
).Methods(http.MethodPost)

// GET /api/employees/{id} - Get employee by ID (all authenticated users)
employeeRouter.HandleFunc("/{id:[0-9]+}",
employeeHandler.GetEmployee).Methods(http.MethodGet)

// GET /api/employees/{id}/details - Get detailed employee information (all
authenticated users)
employeeRouter.HandleFunc("/{id:[0-9]+}/details",
employeeHandler.GetEmployeeDetails).Methods(http.MethodGet)

// PUT /api/employees/{id} - Update employee (admin/manager only)
employeeRouter.Handle("/{id:[0-9]+}",
authMiddleware.RequireRole("admin", "manager")
(http.HandlerFunc(employeeHandler.UpdateEmployee)),
).Methods(http.MethodPut)

// DELETE /api/employees/{id} - Delete employee (admin only)
employeeRouter.Handle("/{id:[0-9]+}",
authMiddleware.RequireRole("admin")
(http.HandlerFunc(employeeHandler.DeleteEmployee)),
).Methods(http.MethodDelete)

// PUT /api/employees/{id}/status - Update employee status (admin/manager only)
```

```
employeeRouter.Handle("/{id:[0-9]+}/status",
authMiddleware.RequireRole("admin", "manager")
(http.HandlerFunc(employeeHandler.UpdateEmployeeStatus)),
).Methods(http.MethodPut)

// GET /api/employees/department/{department_id} - Get employees by department
(all authenticated users)
employeeRouter.HandleFunc("/department/{department_id:[0-9]+}",
employeeHandler.GetEmployeesByDepartment).Methods(http.MethodGet)
}
```